

1 Introdução

Os sistemas operativos modernos distribuem carga entre núcleos de CPU recorrendo a heurísticas clássicas que, apesar de eficazes em produção, tomam decisões subótimas em cenários de elevada contenção de recursos.

Em paralelo, métodos de otimização quântica, nomeadamente o Quantum Approximate Optimization Algorithm (QAOA) [3], têm sido investigados como abordagens alternativas para problemas combinatoriais formuláveis como QUBO (*Quadratic Unconstrained Binary Optimization*) [6, 4].

No entanto, a era NISQ (*Noisy Intermediate-Scale Quantum*) impõe limitações de escala e ruído que tornam difícil avaliar quando um escalonador quântico oferece vantagens concretas sobre alternativas clássicas [8]. Preskill antecipa que esta barreira será ultrapassada com hardware da ordem do *megaquop* — máquinas capazes de executar 10^6 operações quânticas com erro tolerável [9] — fornecendo um referencial concreto para situar a relevância futura deste tipo de abordagem.

Esta tese descreve o desenvolvimento de uma plataforma de testes híbrida quantum-clássica para o problema de atribuição de processos a núcleos de CPU. A plataforma não substitui o escalonador do kernel; isola a decisão de atribuição processo-núcleo, modela-a como um QUBO, resolve-a com QAOA via PennyLane [1], e valida os resultados contra um *solver* exato de *brute force* e simulated annealing [5]. O objetivo não é demonstrar superioridade quântica, mas caracterizar o comportamento do QAOA neste domínio em função da escala do problema.

1.1 Formulação do Problema

Dado um conjunto de processos com utilização de CPU conhecida e um número fixo de núcleos, o sistema deve atribuir cada processo a exatamente um núcleo. O objetivo de otimização é o equilíbrio de carga: a carga total de cada núcleo deve aproximar-se o mais possível da carga média global. A solução deve ainda respeitar restrições de atribuição exclusiva — nenhum processo pode ficar por atribuir ou ser atribuído a múltiplos núcleos simultaneamente.

Expressar estas restrições e este objetivo numa formulação QUBO, compatível com optimizadores quânticos, constitui o problema central desta tese.

2 Formulação QUBO para Escalonamento

2.1 CPU Scheduling em Sistemas Operativos

Um escalonador de CPU determina que processo ou *thread* executa em cada núcleo do processador em cada instante. Escalonadores como o Completely Fair Scheduler (CFS) do Linux, consideram simultaneamente latência, *throughput*, prioridade, afinidade de núcleo e localização de cache, tomando decisões em microssegundos com base em heurísticas clássicas [10].

No entanto, a decisão de atribuição de processos a núcleos é fundamentalmente um problema de partição combinatorial. Dado um conjunto de N processos com cargas w_i e K núcleos, o número de atribuições possíveis cresce como K^N , tornando a procura exaustiva para instâncias reais.

Esta tese adota uma formulação simplificada deste problema: cada *workload entity* possui uma utilização de CPU conhecida, e o objetivo é distribuí-las pelos núcleos disponíveis minimizando o desequilíbrio de carga. Esta abstração isola a componente combinatorial do escalonamento sem replicar a complexidade de um escalonador de kernel completo.

2.2 Formulação QUBO

Um problema QUBO (*Quadratic Unconstrained Binary Optimization*) [6, 4] consiste em minimizar uma expressão quadrática sobre variáveis binárias $x_i \in \{0, 1\}$, representada de forma compacta como:

$$\min_{\mathbf{x} \in \{0,1\}^n} \mathbf{x}^\top Q \mathbf{x} = \sum_i \sum_j Q_{ij} \cdot x_i \cdot x_j \quad (1)$$

onde $Q \in \mathbb{R}^{n \times n}$ é uma matriz cujos coeficientes codificam o problema a resolver. Expandindo o somatório, surgem dois tipos de termos:

- **Termos diagonais** ($i = j$): como $x_i^2 = x_i$ para variáveis binárias, $Q_{ii} \cdot x_i \cdot x_i = Q_{ii} \cdot x_i$. Estes termos associam um custo individual a cada variável — um coeficiente $Q_{ii} < 0$ incentiva o otimizador quântico a escolher $x_i = 1$; um coeficiente $Q_{ii} > 0$ incentiva $x_i = 0$.

- **Termos fora da diagonal** ($i \neq j$): $Q_{ij} \cdot x_i \cdot x_j$ contribui para a energia apenas quando $x_i = x_j = 1$. Um coeficiente positivo penaliza a ativação simultânea de x_i e x_j ; um coeficiente negativo recompensa-a.

A energia de uma solução $\mathbf{x}$ é o valor $\mathbf{x}^\top Q \mathbf{x}$. A solução ótima $\mathbf{x}^*$ é a atribuição binária que minimiza esta energia no espaço de 2^n configurações possíveis.

2.3 Construção da Matriz Q

Dado um conjunto de N processos com cargas w_i e K núcleos, define-se uma variável binária $x_{ij} \in \{0, 1\}$ para cada par processo-núcleo. Esta variável toma o valor 1 quando o processo i é atribuído ao núcleo j , e o valor 0 caso contrário. Na implementação, a variável x_{ij} é armazenada no índice linear $iK + j$ da matriz Q .

O objetivo original do problema é equilibrar a carga entre núcleos. Para isso, mede-se o desequilíbrio através do desvio quadrático total das cargas relativamente à carga média:

$$\min \sum_{j=0}^{K-1} (L_j - L_{avg})^2. \quad (2)$$

onde

$$L_{avg} = \frac{W_{total}}{K}. \quad (3)$$

Para obter a formulação QUBO, este termo é expandido e combinado com penalidades que impõem atribuições válidas. O resultado é codificado na matriz Q , de modo que o *solver* minimize a energia

$$E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x}. \quad (4)$$

Assim, minimizar a energia QUBO corresponde, por construção, a procurar uma atribuição simultaneamente equilibrada e válida.

Para cada núcleo j , a carga total é dada por:

$$L_j = \phi_j + \sum_{i=0}^{N-1} w_i x_{ij}, \quad (5)$$

onde ϕ_j representa carga já comprometida nesse núcleo, relevante para a secção da decomposição iterativa. Na primeira iteração, ou num QUBO sem carga fixa, tem-se $\phi_j = 0$.

Substituindo L_j no termo de um núcleo e usando $x_{ij}^2 = x_{ij}$, obtém-se:

$$\begin{aligned} (L_j - L_{avg})^2 &= \left(\phi_j + \sum_{i=0}^{N-1} w_i x_{ij} - L_{avg} \right)^2 \\ &= \underbrace{\sum_{i=0}^{N-1} d_{ij} x_{ij}}_{\text{termos diagonais}} + 2 \underbrace{\sum_{0 \leq i < k}^{N-1} w_i w_k x_{ij} x_{kj}}_{\text{termos de co-localização}} + \underbrace{(\phi_j - L_{avg})^2}_{\text{constante}}, \end{aligned} \quad (6)$$

onde

$$d_{ij} = w_i^2 + 2w_i \phi_j - 2L_{avg} w_i. \quad (7)$$

Estes coeficientes são então adicionados à matriz Q . Os termos diagonais estão associados à ativação individual de uma variável x_{ij} ; os termos de co-localização aparecem quando dois processos distintos são atribuídos ao mesmo núcleo.

Além do objetivo de balanceamento, é necessário garantir que cada processo é atribuído a exatamente um núcleo. Para isso adiciona-se a penalidade one-hot:

$$P \sum_{i=0}^{N-1} \left(\sum_{j=0}^{K-1} x_{ij} - 1 \right)^2. \quad (8)$$

Esta penalidade segue a prática comum de converter restrições de igualdade em termos quadráticos penalizados em QUBO [4]. Neste caso, contribui com $-P$ na diagonal de cada variável e com $+P$ entre variáveis do mesmo processo em núcleos diferentes.

Combinando o objetivo de balanceamento e a penalidade one-hot, cada entrada da matriz é determinada pelo tipo de relação entre as duas variáveis consideradas:

$$Q[x_{ij}, x_{kl}] = \begin{cases} w_i^2 + 2w_i \phi_j - 2L_{avg} w_i - P, & \text{se } i = k \text{ e } j = l, \\ w_i w_k, & \text{se } i \neq k \text{ e } j = l, \\ P, & \text{se } i = k \text{ e } j \neq l, \\ 0, & \text{se } i \neq k \text{ e } j \neq l. \end{cases} \quad (9)$$

A primeira linha corresponde à contribuição individual de cada variável. A segunda linha penaliza processos distintos que partilham o mesmo núcleo,

enquanto a terceira corresponde à penalidade one-hot entre núcleos diferentes do mesmo processo. A última linha indica que variáveis de processos e núcleos diferentes não interagem diretamente.

2.4 Exemplo Ilustrativo

Considere-se $N = 2$ processos e $K = 2$ núcleos, com cargas $w_0 = 0.3$ e $w_1 = 0.5$, sem carga fixa ($\phi_j = 0$). As variáveis ficam ordenadas como:

índice	0	1	2	3
variável	x_{00}	x_{01}	x_{10}	x_{11}

(10)

A carga média é $L_{avg} = (0.3 + 0.5)/2 = 0.4$.

Antes de adicionar a penalidade one-hot, os termos diagonais do balanceamento são:

$$Q_{0,0}^{bal} = Q_{1,1}^{bal} = w_0^2 - 2L_{avg}w_0 = 0.09 - 2(0.4)(0.3) = -0.15, \quad (11)$$

$$Q_{2,2}^{bal} = Q_{3,3}^{bal} = w_1^2 - 2L_{avg}w_1 = 0.25 - 2(0.4)(0.5) = -0.15. \quad (12)$$

Para o núcleo 0, a expansão de $(L_0 - L_{avg})^2$ contém o termo de co-localização $2w_0w_1x_{00}x_{10} = 0.30x_{00}x_{10}$; de forma análoga, a expansão de $(L_1 - L_{avg})^2$ contém $2w_0w_1x_{01}x_{11} = 0.30x_{01}x_{11}$.

Assim, a matriz do objetivo de balanceamento é:

$$Q_{bal} = \begin{pmatrix} -0.15 & 0 & 0.15 & 0 \\ 0 & -0.15 & 0 & 0.15 \\ 0.15 & 0 & -0.15 & 0 \\ 0 & 0.15 & 0 & -0.15 \end{pmatrix}. \quad (13)$$

Com $P = 1.0$, a penalidade one-hot subtrai 1.0 a cada termo diagonal e adiciona 1.0 entre as variáveis do mesmo processo em núcleos distintos: x_{00} com x_{01} , e x_{10} com x_{11} . A matriz final fica:

$$Q = \begin{pmatrix} -1.15 & 1.0 & 0.15 & 0 \\ 1.0 & -1.15 & 0 & 0.15 \\ 0.15 & 0 & -1.15 & 1.0 \\ 0 & 0.15 & 1.0 & -1.15 \end{pmatrix}. \quad (14)$$

Para verificar o efeito dos termos de co-localização, comparem-se duas soluções viáveis: A, com os processos distribuídos ($x_{00} = x_{11} = 1$), e B, com ambos os processos no núcleo 0 ($x_{00} = x_{10} = 1$). Usando $E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x}$:

$$E_A = Q_{0,0} + Q_{3,3} = -1.15 + (-1.15) = -\mathbf{2.30}, \quad (15)$$

$$E_B = Q_{0,0} + Q_{2,2} + Q_{0,2} + Q_{2,0} = -1.15 + (-1.15) + 0.15 + 0.15 = -\mathbf{2.00}. \quad (16)$$

Como $E_A < E_B$, a solução distribuída tem menor energia. A matriz penaliza, portanto, a concentração dos dois processos no mesmo núcleo, favorecendo uma distribuição mais equilibrada.

Depois de construída a matriz Q , o problema de escalonamento passa a estar expresso numa forma compatível com otimizadores QUBO. O próximo passo será resolver esta matriz, isto é, procurar a bitstring $\mathbf{x}$ que minimiza a energia $E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x}$. Neste trabalho, essa etapa é realizada com QAOA [3], implementado em PennyLane [1].

3 Resolução do QUBO com QAOA

O QAOA é um algoritmo variacional híbrido para otimização combinatória, no qual um circuito quântico é combinado com um otimizador clássico para minimizar o valor esperado de um Hamiltoniano de custo [3, 2]. Neste projeto, a implementação do circuito e da otimização é feita com PennyLane [1].

Importa realçar que a execução foi realizada em simulação, e não em hardware quântico real. Esta opção permite evitar a latência associada aos dispositivos quânticos remotos, bem como os efeitos de ruído físico. Na implementação, o solver tenta usar o backend `lightning.gpu` do PennyLane quando disponível, recorrendo a `lightning.qubit` como alternativa quando a execução em GPU não é possível.

A abordagem implementada atribui um qubit lógico a cada par processo-núcleo. Assim, uma instância com N processos e K núcleos requer $N \cdot K$ qubits.

3.1 Conversão do QUBO em Hamiltoniano

O QAOA trabalha com um Hamiltoniano de custo H_C e não diretamente com a matriz Q . Assim, a energia QUBO

$$E(\mathbf{x}) = \mathbf{x}^\top Q \mathbf{x} \quad (17)$$

tem de ser convertida em forma de operadores quânticos. Esta conversão segue a relação usual entre variáveis binárias e variáveis de *spin* usada em formulações

Ising de problemas combinatórios [7]. Para isso, cada variável binária $x_i \in \{0, 1\}$ é mapeada para operadores de Pauli Z_i usando a substituição:

$$x_i = \frac{I - Z_i}{2}. \quad (18)$$

Esta substituição permite representar os valores binários 0 e 1 através dos dois estados da base computacional, $|0\rangle$ e $|1\rangle$. Quando o qubit está no estado $|0\rangle$, o operador Z_i tem valor $+1$, resultando em $x_i = 0$; quando está no estado $|1\rangle$, o operador Z_i tem valor -1 , resultando em $x_i = 1$.

Assim, um termo linear da diagonal do QUBO transforma-se em:

$$Q_{ii}x_i = Q_{ii}\frac{I - Z_i}{2} = \frac{Q_{ii}}{2}I - \frac{Q_{ii}}{2}Z_i. \quad (19)$$

Para os termos quadráticos, considerando $q_{ij} = Q_{ij} + Q_{ji}$, obtém-se:

$$q_{ij}x_ix_j = q_{ij}\frac{I - Z_i}{2}\frac{I - Z_j}{2} = \frac{q_{ij}}{4}(I - Z_i - Z_j + Z_iZ_j). \quad (20)$$

Desta forma, o Hamiltoniano de custo pode ser escrito como:

$$H_C = cI + \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j, \quad (21)$$

onde c é uma constante, h_i são os coeficientes lineares associados a cada qubit, e J_{ij} representa a interação entre pares de qubits.

Uma vez obtido o Hamiltoniano de custo H_C este passa a definir a energia associada a cada estado. No QAOA, esta informação é usada através de uma operação unitária de custo:

$$U_C(\gamma) = e^{-i\gamma H_C}, \quad (22)$$

onde γ é um parâmetro variacional. Esta operação pode ser vista como uma evolução temporal sob o Hamiltoniano H_C durante um tempo controlado por γ . Como H_C é diagonal na base computacional, cada bitstring recebe uma fase dependente da sua energia:

$$U_C(\gamma)|x\rangle = e^{-i\gamma E(x)}|x\rangle. \quad (23)$$

Logo a camada de custo não altera diretamente a probabilidade de cada bitstring, ela altera as suas fases relativas. Essas fases tornam-se úteis quando é aplicada a camada de mistura, responsável por distribuir amplitude entre soluções.

A camada de mistura é definida por outro Hamiltoniano H_M e pela unidade

$$U_M(\beta) = e^{-i\beta H_M}, \quad (24)$$

onde β é também um parâmetro variacional. Enquanto U_C traduz informação sobre a qualidade das soluções através das fases, U_M permite explorar novas soluções ao misturar amplitudes entre estados da base computacional.

3.2 Mixer X e XY

O Hamiltoniano H_M presente em (24) é chamado de *mixer* e temos disponível dois tipos: X e XY.

A diferença entre eles essencialmente afeta se o QAOA explora todo o espaço binário ou apenas o espaço de atribuições válidas.

3.2.1 Mixer X

Este é o padrão do QAOA. O Hamiltoniano de forma simplificada é:

$$H_M^X = \sum_i X_i \quad (25)$$

onde X_i é o operador de Pauli X aplicado ao qubit i .

Este mixer define que o estado inicial deve ser a superposição uniforme sobre todos os bitstrings:

$$|s\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle \quad (26)$$

No entanto, este espaço contém muitos estado inválidos. Por exemplo, com $N = 1$ e $K = 2$ os estados possíveis são:

$$00, 01, 10, 11 \quad (27)$$

No entanto só dois são válidos:

$$01, 10 \quad (28)$$

E por essa razão que para o nosso caso de uso, o *mixer* XY é o mais relevante.

3.2.2 Mixer XY

Em vez de operar no espaço total de soluções, ele opera somente nos qubits que representam os possíveis núcleos desse processo.

Para cada processo i , há um bloco de K qubits:

$$x_{i0}, x_{i1}, \dots, x_{i,K-1}, \quad (29)$$

O mixer XY atua somente entre pares de qubits desse mesmo bloco através da operação:

$$X_{ij}X_{ik} + Y_{ij}Y_{ik} \quad (30)$$

que permite trocar amplitude entre estados como:

$$|10\rangle \leftrightarrow |01\rangle \quad (31)$$

mas sem criar os estados inválidos

$$|00\rangle < - > |11\rangle \tag{32}$$

Dito isto, concluímos que o mixer XY é o mais válido para este projeto, transformando um espaço de soluções de tamanho

$$2^{N \cdot K}$$

num espaço de tamanho

$$K^N$$

Referências

- [1] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, Shahnawaz Ahmed, Vishnu Ajith, M. Sohaib Alam, Guillermo Alonso-Linaje, B. AkashNarayanan, Ali Asadi, et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations, 2018.
- [2] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3:625–644, 2021.
- [3] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm, 2014.
- [4] Fred Glover, Gary Kochenberger, and Yu Du. A tutorial on formulating and using QUBO models, 2018.
- [5] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [6] Gary Kochenberger, Jin-Kao Hao, Fred Glover, Mark Lewis, Zhipeng Lü, Haibo Wang, and Yang Wang. The unconstrained binary quadratic programming problem: A survey. *Journal of Combinatorial Optimization*, 28(1):58–81, 2014.

- [7] Andrew Lucas. Ising formulations of many NP problems. *Frontiers in Physics*, 2:5, 2014.
- [8] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.
- [9] John Preskill. Beyond NISQ: The megaquop machine, 2025.
- [10] The Linux Kernel Development Community. CFS scheduler. <https://docs.kernel.org/scheduler/sched-design-CFS.html>. Accessed: 2026-06-18.